

Foundations of Adam optimizer

[Adam optimizer]

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

1.0 Content Block

Adam (short for Adaptive Moment Estimation) is a 2014 update to the RMSProp optimizer combining it with the main feature of the Momentum method. In this optimization algorithm, running averages with exponential forgetting of both the gradients and the second moments of the gradients are used. Given parameters $w(t)$ and a loss function $L(t)$, where t indexes the current training iteration (indexed at 1), Adam's parameter update is given by:

2.0 Content Block

Averaging Averaged stochastic gradient descent, invented independently by Ruppert and Polyak in the late 1980s, is ordinary stochastic gradient descent that records an average of its parameter vector over time. That is, the update is the same as for ordinary stochastic gradient descent, but the algorithm also keeps track of

3.0 Content Block

$$w_{\text{new}} = w_{\text{old}} + \eta \frac{1}{1 + \eta \sum_{i=1}^t (y_i - x_i' w_{\text{old}})^2} (y_i - x_i' w_{\text{old}}) x_i$$

4.0 Content Block

Linear regression Suppose we want to fit a straight line $\hat{y} = w_1 + w_2 x$ to a training set with observations $((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$ and corresponding estimated responses $(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$ using least squares. The objective function to be minimized is

5.0 Content Block

AdaGrad AdaGrad (for adaptive gradient algorithm) is a modified stochastic gradient descent algorithm with per-parameter learning rate, first published in 2011. Informally, this increases the learning rate for sparser parameters and decreases the learning rate for ones that are less sparse. This strategy often improves convergence performance over standard stochastic gradient descent in settings where data is sparse and sparse parameters are more informative. Examples of such applications include natural language processing and image recognition. It still has a base learning rate η , but this is multiplied with the elements of a vector $\{G_{j,i}\}$ which is the diagonal of the outer product matrix

6.0 Content Block

where, γ is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

7.0 Content Block

As the algorithm sweeps through the training set, it performs the above update for each training sample. Several passes can be made over the training set until the algorithm converges. If this is done, the data can be shuffled for each pass to prevent cycles. Typical implementations may use an adaptive learning rate so that the algorithm converges. In pseudocode, stochastic gradient descent can be presented as :

8.0 Content Block

where i is uniformly sampled between 1 and n . Although theoretical convergence of this procedure happens under relatively mild assumptions, in practice the procedure can be quite unstable. In particular, when η is misspecified so that $I - \eta X^T X$ has large absolute eigenvalues with high probability, the procedure may diverge numerically within a few iterations. In contrast, implicit stochastic gradient descent (shortened as ISGD) can be solved in closed-form as:

9.0 Content Block

for $\Sigma(w) = \frac{1}{n^2} \left(\sum_{i=1}^n Q_i(w) - Q(w) \right) \left(\sum_{i=1}^n Q_i(w) - Q(w) \right)^T$ where dB_t denotes the Ito-integral with respect to a Brownian motion is a more precise approximation in the sense that there exists a constant $C > 0$ such that

10.0 Content Block

$w_{\text{new}} := \arg \min_w \left\{ Q_i(w) + \frac{1}{2\eta} \|w - w_{\text{old}}\|^2 \right\}$.

11.0 Content Block

Backtracking line search Backtracking line search is another variant of gradient descent. All of the below are sourced from the mentioned link. It is based on a condition known as the Armijo–Goldstein condition. Both methods allow learning rates to change at each iteration; however, the manner of the change is different. Backtracking line search uses function evaluations to check Armijo's condition, and in principle the loop in the algorithm for determining the learning rates can be long and unknown in advance. Adaptive SGD does not need a loop in determining learning rates. On the other hand, adaptive SGD does not guarantee the "descent property" – which Backtracking line search enjoys – which is that $f(x_{n+1}) \leq f(x_n)$ for all n . If the gradient of the cost function is globally Lipschitz continuous, with Lipschitz constant L , and learning rate is chosen of the order $1/L$, then the standard version of SGD is a special case of backtracking line search.

12.0 Content Block

The scaling factor $\xi^* \in \mathbb{R}$ can be found through the bisection method since in most regular models, such as the aforementioned generalized linear models, function $q()$ is decreasing, and thus the search bounds for ξ^* are $[\min(0, f(0)), \max(0, f(0))]$.

13.0 Content Block

Approximations in continuous time For small learning rate η stochastic gradient descent $(w_n)_{n \in \mathbb{N}}$ can be viewed as a discretization of the gradient

flow ODE

14.0 Content Block

where $x_j' w = x_{j,1} w_1 + x_{j,2} w_2 + \dots + x_{j,p} w_p$ $\{\displaystyle x_{j,1} w_1 + x_{j,2} w_2 + \dots + x_{j,p} w_p\}$ indicates the inner product. Note that x $\{\displaystyle x\}$ could have "1" as the first element to include an intercept. Classical stochastic gradient descent proceeds as follows: